

Documentação da Migração de Sistemas para Python

1. Introdução

Este trabalho apresenta a documentação do processo de migração de sistemas desenvolvidos originalmente em Java, C e COBOL para a linguagem Python. A proposta principal foi manter a mesma lógica dos programas originais, mas adaptar sua estrutura para uma linguagem mais simples, moderna e de fácil leitura.

Durante a migração, foi possível perceber que, mesmo mudando a linguagem de programação, a lógica principal dos sistemas permaneceu a mesma. O que mudou foi a forma de escrever o código, organizar os dados, receber informações do usuário e controlar o funcionamento dos programas.

A escolha do Python trouxe impactos positivos para os sistemas, principalmente na simplificação da sintaxe, na redução da quantidade de código e na modernização da estrutura dos programas.

2. Objetivo da Migração

O objetivo da migração foi reescrever sistemas feitos em Java, C e COBOL utilizando Python, mantendo suas funcionalidades principais.

Além disso, a migração permitiu comparar como diferentes linguagens resolvem problemas parecidos. Em linguagens como Java, C e COBOL, é comum utilizar estruturas mais rígidas, declarar tipos de variáveis e escrever mais comandos para realizar tarefas simples. Já em Python, muitas dessas etapas são feitas de forma mais direta.

Com isso, a migração ajudou a demonstrar que Python pode tornar o código mais legível, mais curto e mais fácil de manter, sem alterar a lógica essencial do sistema.

3. Sistema de Alunos — Migração de Java para Python

O Sistema de Alunos foi originalmente desenvolvido em Java. Ele possui um menu com opções para adicionar aluno, listar alunos, buscar a nota de um aluno e encerrar o programa.

Na versão em Java, foram utilizados recursos como Scanner para entrada de dados, HashMap para armazenar as notas e ArrayList para guardar os nomes dos

alunos. O controle do menu era feito por meio de uma estrutura switch, que direcionava o programa de acordo com a opção escolhida pelo usuário.

Na versão em Python, essa estrutura foi simplificada. Em vez de utilizar HashMap e ArrayList, foi usada uma lista chamada alunos. Cada aluno pode ser representado por um dicionário, contendo informações como nome e nota.

A entrada de dados também ficou mais simples. O Scanner do Java foi substituído pelo input() do Python. Para valores numéricos, como a nota do aluno, foi utilizada a conversão com float().

Outro ponto positivo foi o tratamento de erros com try/except, que permite evitar que o programa pare quando o usuário digita uma informação inválida.

Os métodos da versão em Java foram adaptados para funções em Python, como adicionar_Alunos(), listar_Alunos() e buscar_Notas(). Dessa forma, a organização do código foi mantida, mas com uma sintaxe mais simples.

O impacto positivo dessa migração foi a redução da complexidade. O sistema continuou com as mesmas funções principais, mas passou a ter uma estrutura mais enxuta e mais fácil de entender.

4. Sistema de Votação — Migração de Java para Python

O Sistema de Votação também foi originalmente desenvolvido em Java. Ele apresenta opções de voto, permite que o usuário escolha uma delas e contabiliza os votos até que o usuário digite 0 para encerrar.

Na versão em Java, foram utilizados arrays para guardar as opções e a quantidade de votos. Também foi usada uma variável booleana para controlar se a votação continuava ativa. A entrada de dados era feita com Scanner, e a validação utilizava hasNextInt().

Na versão em Python, os arrays foram substituídos por listas. A lista opcoes armazena as opções disponíveis, enquanto a lista votos armazena a quantidade de votos de cada opção.

A lógica foi mantida de forma muito parecida. O usuário escolhe uma opção, o programa verifica se ela é válida e soma um voto na posição correspondente da lista. O uso de escolha-1 foi mantido para ajustar a escolha do usuário ao índice da lista, já que a contagem dos índices começa em 0.

A validação de entrada passou a ser feita com try/except. Caso o usuário digite algo que não seja número, o sistema mostra uma mensagem de erro e continua funcionando.

Nesse caso, o principal impacto positivo foi a simplificação do código. Python permitiu escrever a mesma lógica com menos comandos, sem a necessidade de declarar tipos de variáveis ou fechar objetos de entrada, como acontece no Java.

5. Sistema de Funcionários — Migração de C para Python

O Sistema de Funcionários foi originalmente desenvolvido em linguagem C. Ele possui opções para adicionar funcionário, listar funcionários, buscar funcionário e encerrar o programa.

Na versão em C, foi utilizada uma struct chamada `Funcionario`, contendo os campos `nome`, `cargo` e `salario`. Também foi usado um vetor de tamanho fixo para armazenar os funcionários, além de uma variável total para controlar a quantidade de cadastros realizados.

A linguagem C exige um controle maior sobre os dados. Por exemplo, foi necessário usar `scanf`, `fgets`, `getchar()` e `strcspn()` para entrada de dados e tratamento de strings.

Na versão em Python, a struct foi substituída por um dicionário. Cada funcionário passou a ser representado com chaves como `nome`, `cargo` e `salario`.

O vetor fixo do C foi substituído por uma lista dinâmica chamada `funcionarios`. Isso trouxe uma melhoria importante, pois em Python não é necessário definir previamente um limite fixo de registros. Novos funcionários podem ser adicionados com o método `append()`.

A variável total também deixou de ser necessária, já que a própria lista controla a quantidade de itens cadastrados.

Outra simplificação ocorreu na busca. Em C, era necessário utilizar `strcmp()` para comparar strings. Em Python, essa comparação pode ser feita diretamente com `==`.

O impacto positivo dessa migração foi bem claro: o sistema ficou mais simples, mais flexível e com menos necessidade de controle manual. Python reduziu a quantidade de detalhes técnicos necessários para realizar as mesmas operações.

6. Sistema de Cadastro de Alunos — Migração de COBOL para Python

O sistema desenvolvido originalmente em COBOL tinha como objetivo cadastrar alunos, armazenar suas notas, calcular a média da turma e listar os dados cadastrados.

COBOL é uma linguagem mais antiga e possui uma estrutura mais extensa e formal. Normalmente, seus programas são organizados em divisões, com áreas

específicas para identificação, dados e procedimentos. Além disso, as variáveis costumam ser declaradas com tamanhos e formatos definidos previamente.

Na versão em Python, essa estrutura foi bastante simplificada. Foram utilizadas listas para armazenar os nomes e as notas dos alunos. A quantidade de alunos foi definida por uma variável chamada `num_alunos`, e o cadastro foi realizado com um laço `for`.

O cálculo da média foi separado em uma função chamada `media()`, e a listagem dos alunos foi feita por meio da função `lista_alunos()`.

Essa divisão deixou o código mais organizado e fácil de entender. Em vez de uma estrutura extensa, Python permitiu representar a mesma lógica com poucas funções e comandos diretos.

O principal impacto positivo dessa migração foi a modernização do código. A versão em Python ficou mais próxima de uma linguagem atual, com leitura mais intuitiva e manutenção mais simples.

7. Principais Mudanças Observadas

A principal mudança observada foi a simplificação da sintaxe. Em Python, não foi necessário declarar tipos de variáveis, criar estruturas muito rígidas ou escrever muitos comandos para tarefas básicas.

Outra mudança importante foi o uso de listas e dicionários. Estruturas como arrays, vetores, `ArrayList`, `HashMap` e `struct` foram adaptadas para estruturas mais simples e flexíveis do Python.

A entrada de dados também ficou mais direta. Enquanto Java utiliza `Scanner`, C utiliza `scanf` e `fgets`, e COBOL possui comandos próprios, Python utiliza principalmente `input()`.

O tratamento de erros com `try/except` também foi um ponto positivo, pois tornou o código mais seguro contra entradas inválidas.

Além disso, a organização em funções ajudou a manter os programas mais limpos. Cada parte do sistema ficou responsável por uma tarefa específica, como adicionar, listar, buscar ou calcular.

8. Impactos Positivos da Migração para Python

A migração para Python trouxe vários benefícios para os sistemas analisados.

O primeiro benefício foi a redução da quantidade de código. Python permitiu escrever menos linhas para realizar as mesmas tarefas, tornando os programas mais objetivos.

Outro impacto positivo foi a melhora na legibilidade. A sintaxe do Python é mais próxima da linguagem natural, o que facilita a leitura tanto para quem escreveu o código quanto para outras pessoas que precisem entendê-lo depois.

Também houve modernização da estrutura dos sistemas. Ao substituir estruturas mais antigas ou mais rígidas por listas, dicionários e funções simples, os programas ficaram mais adaptados a uma linguagem atual e bastante utilizada no mercado.

A manutenção do código também se tornou mais fácil. Como os sistemas ficaram mais curtos e organizados, futuras alterações podem ser feitas com menos dificuldade.

Além disso, Python reduziu a necessidade de controle manual presente em linguagens como C, principalmente no tratamento de strings, tamanho de vetores e manipulação de entrada de dados.

De forma geral, a migração tornou os sistemas mais simples, modernos, legíveis e práticos para fins acadêmicos e para futuras melhorias.

9. Dificuldades Encontradas na Migração

Apesar dos benefícios, algumas adaptações foram necessárias durante a migração.

Uma das dificuldades foi transformar as estruturas de dados de cada linguagem para Python. Por exemplo, a struct do C foi adaptada para dicionário, enquanto o HashMap e o ArrayList do Java foram substituídos por listas e dicionários.

Também foi necessário adaptar a forma de entrada de dados. Em Python, o input() sempre recebe os dados como texto, então foi preciso converter algumas entradas para int ou float.

Outro ponto importante foi o cuidado com os índices das listas. Assim como em Java e C, Python começa a contagem em 0. Por isso, no sistema de votação, foi necessário manter o ajuste com escolha-1.

Além disso, alguns detalhes de sintaxe exigem atenção, como o uso correto de aspas em f-strings e a chamada correta de métodos, como capitalize().

10. Conclusão

A migração dos sistemas para Python mostrou que é possível manter a lógica principal de um programa mesmo mudando a linguagem de programação.

Os sistemas continuaram executando as mesmas funções, mas passaram a ter uma estrutura mais simples, moderna e legível. Python facilitou a escrita do

código, reduziu a quantidade de comandos necessários e tornou os programas mais fáceis de entender.

Comparando com Java, C e COBOL, Python apresentou vantagens principalmente na simplicidade, na flexibilidade das estruturas de dados e na facilidade de manutenção.

Portanto, a migração foi positiva porque permitiu modernizar os sistemas sem perder suas funcionalidades originais. Além disso, ajudou a compreender melhor as diferenças entre linguagens de programação e reforçou que a lógica de programação pode ser aplicada em diferentes contextos, mesmo quando a sintaxe muda.